

Разработка языка программирования Weasel



Студент группы ПС-31
Глухарев Степан

Научный руководитель
Нехорошкова Людмила Георгиевна

Формулировка задания

Разработать экспериментальный язык программирования и инструменты его исполнения — байткод-компилятор и виртуальную машину.

Была идея...

1. Система типов для Clight

Скалярные типы данных:

s bitten - целый 8-bit

u bitten - натурал 8-bit

s little - целый 16-bit

u little - натурал 16-bit

s number - целый 32-bit

u number - натурал 32-bit

s longer - целый 64-bit

u longer - натурал 64-bit

single - плавающая точка 32-bit

double - плавающая точка 64-bit

boolen - логический тип true / false

string - строки

voided - пустой тип (для функций)

Математические типы данных:

planar - 2D вектор (направление)

vector - 3D вектор (3D координаты)

quadra - 4D вектор (3D трансформации или цвета) + acolor (алиас)

linear - матрица 3x3 (матрица для линейных преобразований)

matrix - матрица 4x4 (матрица для проекций, трансформаций)

Контейнерные типы данных или че это....:

slices - динамический массив (структура: указатель + длина + вместимость)

buffer - сырая память (толстый указатель: начало + размер)

rpoint - сырой указатель (raw pointer, как T* в C)

fpoint - указатель на функцию

struct - объявление структуры

unions - объявление объединения (исправил)

enumer - объявление перечисления (enum)

Идея

Язык программирования, в котором синтаксис и семантика проектируются вокруг задач описания алгоритмов, а не вокруг технических деталей их реализации.

Цель

Разработка и реализация языка программирования, ориентированного на описание алгоритмов и структур данных, а также создание программного комплекса (тулчейна), обеспечивающего полный цикл написания программ от языка до реализации.

Задачи

[01]

Провести анализ существующих подходов к представлению алгоритмов в языках программирования и сформулировать требования к разрабатываемому языку.

[02]

Разработать основу языка:

- Спроектировать синтаксис;
- Реализовать компилятор языка;
- Разработать виртуальную машину.

[03]

Выполнить апробацию разработанного языка на примере реализации классических алгоритмов и оценить его применимость для их описания.

Исследование и анализ

Исследование псевдокода

Вид псевдокода	Описание
Естественный язык	Алгоритм описывается словами, близкими к человеческой речи
Структурный псевдокод	Используются конструкции IF, WHILE, FOR без привязки к языку
Языкоподобный псевдокод	Очень похож на конкретный язык программирования
Математический псевдокод	Активно используются математические обозначения и формулы

Исследование псевдокода

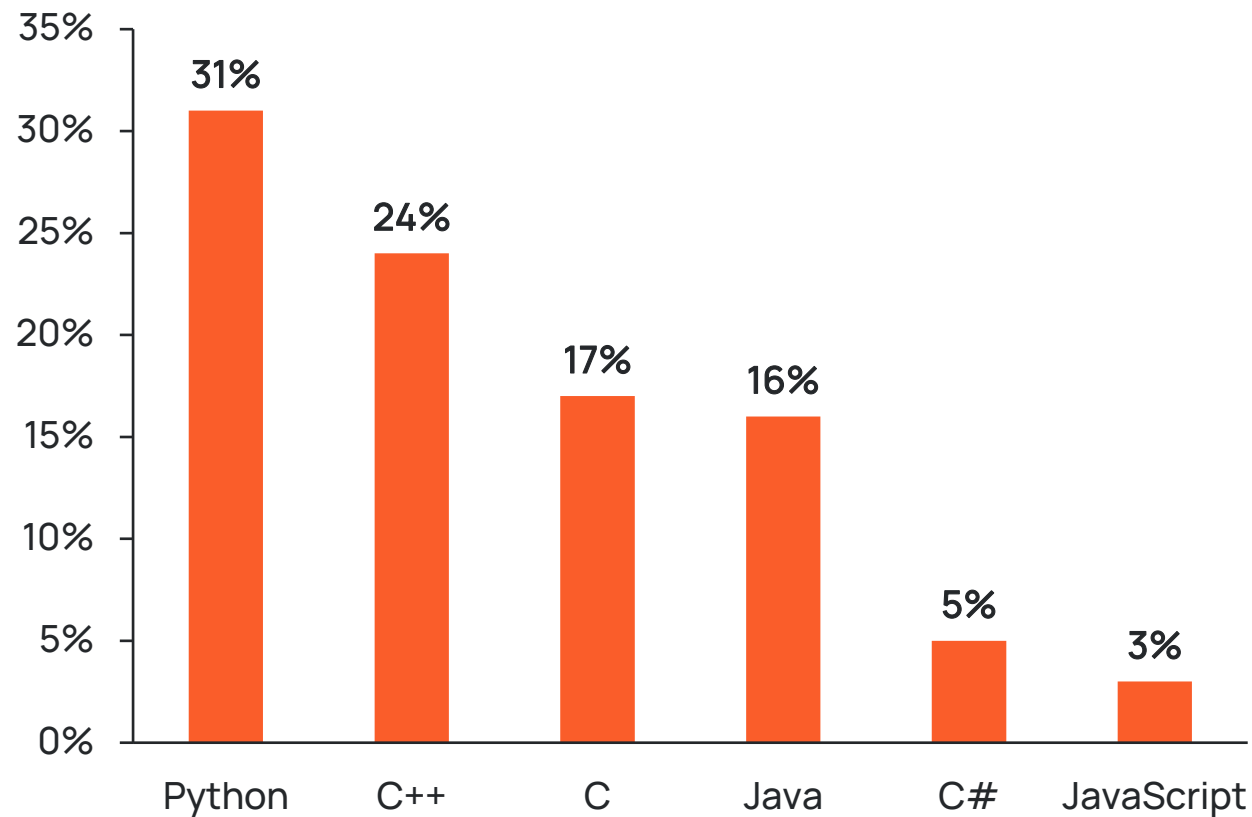
MERGE_SORT(A, p, r)

```
1  if  $p < r$ 
2      then  $q \leftarrow \lfloor (p + r) / 2 \rfloor$ 
3          MERGE_SORT( $A, p, q$ )
4          MERGE_SORT( $A, q + 1, r$ )
5          MERGE( $A, p, q, r$ )
```

BUBBLESORT(A)

```
1  for  $i \leftarrow 1$  to  $\text{length}[A]$ 
2      do for  $j \leftarrow \text{length}[A]$  downto  $i + 1$ 
3          do if  $A[j] < A[j - 1]$ 
4              then Поменять местами  $A[j] \leftrightarrow A[j - 1]$ 
```

Популярность языков для изучения алгоритмов



В других языках

```
void BubbleSort::Sort(std::vector<int>& array)
{
    AssertIsNonNull(array);

    if (array.empty() || array.size() == 1)
    {
        return;
    }

    int n = array.size();

    for (int i = 0; i < n - 1; ++i)
    {
        PassThrough(array, n - i - 1);
    }
}
```

```
def sort(self, array: list[int]) -> None:
    self._assert_is_not_null(array)

    if len(array) <= 1:
        return

    n = len(array)

    for i in range(n - 1):
        self._pass_through(array, n - i - 1)
```

Поиск проблем

```
def something(self, number: int) -> bool:
    if number <= 1:
        return False

    if number == 2:
        return True

    if number % 2 == 0:
        return False

    for i in range(3, int(number ** 0.5) + 1, 2):
        if number % i == 0:
            return False

    return True
```

```
void PassThrough(std::vector<int>& arr, int lastIndex)
{
    for (int i = 0; i < lastIndex; ++i)
    {
        if (ShouldSwap(arr[i], arr[i + 1]))
        {
            Swap(arr[i], arr[i + 1]);
        }
    }
}

void Something(std::vector<int>& array)
{
    AssertIsNonNull(array);

    if (array.empty() || array.size() == 1)
    {
        return;
    }

    int n = array.size();

    for (int i = 0; i < n - 1; ++i)
    {
        PassThrough(array, n - i - 1);
    }
}
```

Поиск проблем

```
def something(self, number: int) -> bool:
    if number <= 1:
        return False

    if number == 2:
        return True

    if number % 2 == 0:
        return False

    for i in range(3, int(number ** 0.5) + 1, 2):
        if number % i == 0:
            return False

    return True
```

```
void PassThrough(std::vector<int>& arr, int lastIndex)
{
    for (int i = 0; i < lastIndex; ++i)
    {
        if (ShouldSwap(arr[i], arr[i + 1]))
        {
            Swap(arr[i], arr[i + 1]);
        }
    }
}

void Something(std::vector<int>& array)
{
    AssertIsNonNull(array);

    if (array.empty() || array.size() == 1)
    {
        return;
    }

    int n = array.size();

    for (int i = 0; i < n - 1; ++i)
    {
        PassThrough(array, n - i - 1);
    }
}
```

Реализация базы языка

Архитектура языка Weasel



Разработка синтаксиса и семантики языка

Тьюринг-полнота языка

Тьюринг-полным называется язык программирования, способный выразить любой вычислимый алгоритм при наличии неограниченных вычислительных ресурсов.

Этапы проектирования языка

Проектирование языка выполнялось поэтапно, начиная с минимального набора возможностей, необходимых для достижения тьюринг-полноты:

- [00] Лексика и литералы
- [01] Типы данных
- [02] Переменные и присваивание
- [03] Арифметические и логические операции
- [04] Управление потоком выполнения
- [05] Функции

Система типов

Строгая статическая типизация:
тип известен на этапе компиляции.

Реализовано явное приведение типов
через нативные функции.

Литералы автоматически подстраиваются
под контекст.

Управление памятью спрятано
от пользователя.

```
val int score = -15
val uint count = 100
val real pi = 3.14159
val bool isReady = true
val string name = "Weasel"

val real scoreReal = score.real()
val real mult = scoreReal * 3

val array[int] data = [1, 2, 3, 4, 5]
val array[int] big = array[int](1000)
```

Переменные

var — изменяемая переменная. Можно переприсвоить в любой момент, инициализатор необязателен — получит нулевое значение типа.

val — имя привязывается к значению один раз при объявлении и больше не меняется. Инициализация обязательна.

let — константа времени компиляции. Компилятор не выделяет стековый слот, а просто подставляет значение на место каждого обращения.

```
var uint counter = 0
counter = counter + 1

val uint maxScore = 100
# maxScore = 99

let string bufferSize = 256
```

Циклы

repeat — цикл с 6 сценариями реализации для очевидности записи.

run() {} и **{ } run ()** — предусловный и постусловный цикл.

break и **continue** применимы в любом виде цикла.

```
repeat i in 0 .. 5
repeat i in 0 .. 10 step 2
repeat i in 5 down 0
repeat n in [1, 2, 3]
repeat i, n in [1, 2, 3]
repeat 3 times
```

```
run (x > 0)
{ }
```

```
{ }
run (x > 0)
```

Сопоставление с шаблоном

when без предмета — замена цепочке **else if**.

when с предметом — замена **switch**, поддерживает несколько значений в одной ветке.

Проверка полноты для **enum** — если добавить новый статус и забыть обновить **when**, программа не скомпилируется.

```
enum Status [ NEW, PROCESSING, DONE, ERROR ]

val Status task = Status.PROCESSING

when (task)
{
    Status.NEW -> println("Создана")
    Status.PROCESSING -> println("Выполняется")
    Status.DONE -> println("Завершена")
    Status.ERROR -> println("Ошибка")
    # else не нужен — компилятор проверяет все ветки
}
```


Функции

Функция — именованный блок кода с явными типами параметров и возвращаемого значения. Типы проверяются строго на всех границах вызовов: неявных преобразований нет.

Параметр может иметь значение по умолчанию — тогда при вызове аргумент можно опустить.

Несколько функций могут носить одно имя, если различаются сигнатурой.

```
int <- sum(val array[int] nums)
{
    var int total = 0
    repeat i in 0 .. nums.size
    {
        total = total + nums[i]
        # nums[i] = 0
    }
    return total
}

void <- fill(var array[int] arr, int value)
{
    repeat i in 0 .. arr.size
    {
        arr[i] = value
    }
}

println(sum([1, 2, 3, 4, 5]))
```

Классы

Классы — основной механизм пользовательских типов.

Синтаксис парсится и регистрируется в системе типов — пользовательские типы работают наравне со встроенными типами.

Класс описывает структуру данных и операции над ней, а не механизм управления памятью.

```
class Matrix
{
  var int rows
  var int cols
  var array[int] data

  int <- get(int i, int j)

  public:
    Matrix(int r, int c)

    int <- numRows()
    int <- numCols()
    void <- set(int i, int j, int value)
    Matrix <- multiply(Matrix other)
    void <- show()
}
```

Дополнительно

Форматирование кода

Для автоматического форматирования исходных файлов .wes реализован инструмент `wesc --format`, встроенный в компилятор Weasel. Форматирование работает на уровне токенов — парсер не задействуется, что обеспечивает устойчивость к синтаксическим ошибкам в обрабатываемом файле.

Стандартная библиотека

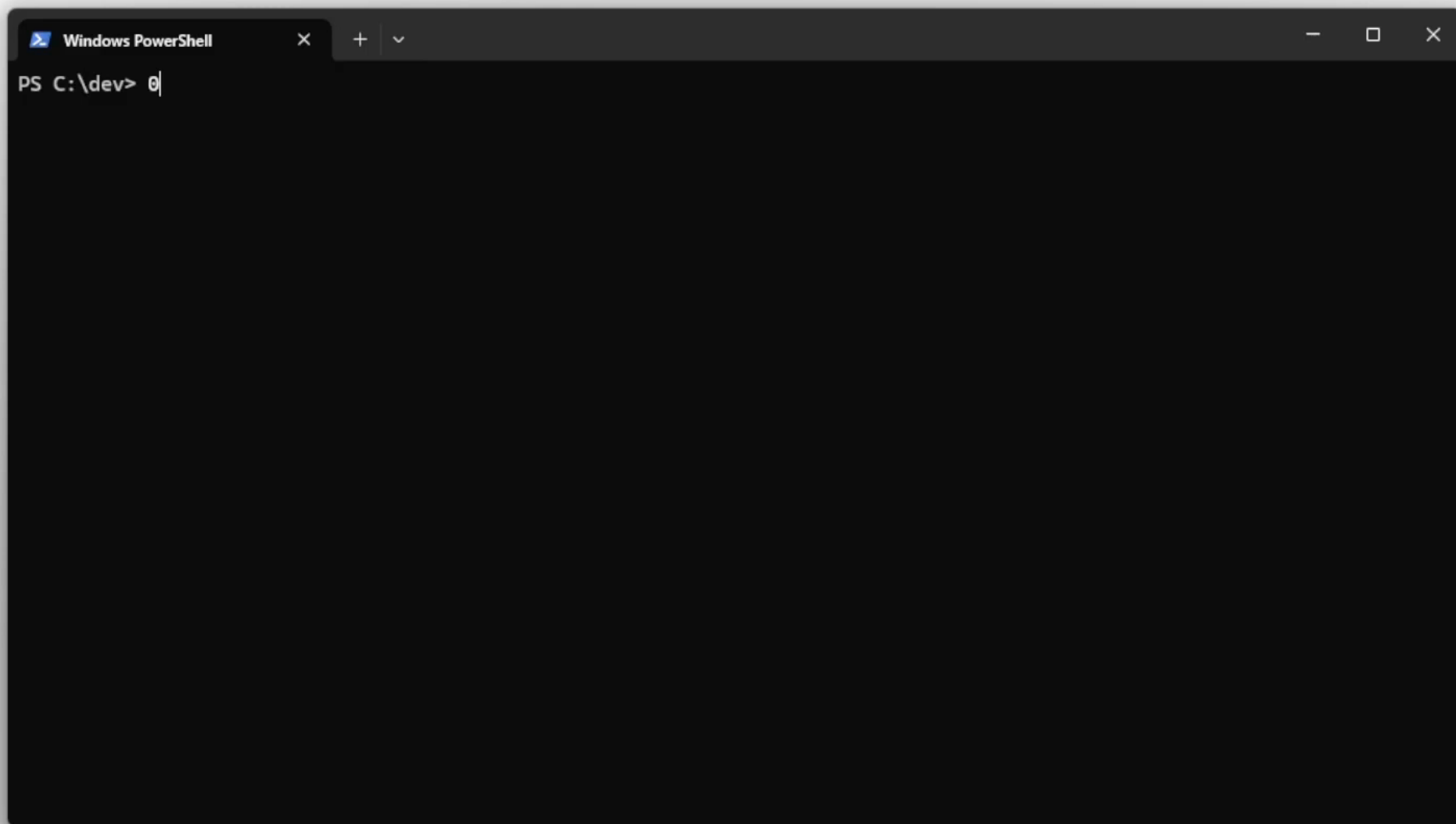
Стандартная библиотека Weasel реализована на самом языке и подключается через механизм импорта. Компилятор при встрече директивы `import` разрешает путь относительно каталога `stdlib`, компилирует модуль и объединяет пространства имён до начала семантического анализа основного файла.

Система сборки

wkit — отдельный инструмент командной строки для управления проектами на Weasel. Работает независимо от системного **make** и знает про специфику языка.

Конфиг написан на самом языке Weasel.

```
wkit new myapp      # создать новый проект
wkit build           # собрать проект
wkit run             # собрать и запустить
wkit format          # отформатировать все .wes файлы
```



Что ещё хотелось бы
реализовать

Хотелось бы реализовать

- Расширения для сред разработки
- Режим отладки по языку
- Кодогенерация второго backend
- Языковые роли

Тестирование языка в деле

```

class Matrix
{
    var int rows
    var int cols
    var array[int] data

    int <- get(int i, int j)
    {
        return data[i * cols + j]
    }

public {
    Matrix(int r, int c)
    {
        rows = r
        cols = c
        data = array[int](r * c)
    }

    int <- numRows()
    {
        return rows
    }

    int <- numCols()
    {
        return cols
    }

    void <- set(int i, int j, int value)
    {
        data[i * cols + j] = value
    }
}

```

```

Matrix <- multiply(Matrix other)
{
    if (cols >< other.numRows())
    {
        println("Ошибка: несовместимые размеры матриц для умножения")
        return Matrix(0, 0)
    }
    var Matrix result = Matrix(rows, other.numCols())
    repeat (i in 0 .. rows)
    {
        repeat (j in 0 .. other.numCols())
        {
            var int sum = 0
            repeat (k in 0 .. cols)
            {
                sum = sum + get(i, k) * other.get(k, j)
            }
            result.set(i, j, sum)
        }
    }
    return result
}

void <- show()
{
    repeat (i in 0 .. rows)
    {
        repeat (j in 0 .. cols)
        {
            print(get(i, j), " ")
        }
        println("")
    }
}

```

```

class IntList
{
    var uint count
    var uint capacity
    var array[int] data

    uint <- nextCapacity()
    {
        var int grown = (capacity + capacity).int() / 2

        if (grown == capacity.int())
        {
            grown = capacity.int() + 1
        }

        return grown
    }

    void <- reallocate(uint newCapacity)
    {
        var array[int] newData = array[int](newCapacity)
        repeat (i in 0 .. count)
        {
            newData[i] = data[i]
        }
        data = newData
        capacity = newCapacity
    }
}

```

```

public {
    IntList()
    {
        count = 0
        capacity = 4
        data = array[int](4)
    }

    IntList(uint initCapacity)
    {
        var uint cap = initCapacity

        if (cap < 1)
        {
            cap = 1
        }

        count = 0
        capacity = cap
        data = array[int](cap)
    }

    uint <- size()
    {
        return count
    }

    bool <- isEmpty()
    {
        return count == 0
    }

    void <- reserve(uint minCapacity)
    {
        if (minCapacity > capacity)
        {
            reallocate(minCapacity)
        }
    }

    void <- push(int value)
    {
        if (count == capacity)
        {
            reallocate(nextCapacity())
        }
        data[count] = value
        count = count + 1
    }

    int <- get(uint index)
    {
        return data[index]
    }

    void <- set(uint index, int value)
    {
        data[index] = value
    }

    int <- pop()
    {
        count = count - 1
        return data[count]
    }
}

```

Пример курсовой работы ПС-21

На некотором поле находятся роботы. Требуется обеспечить движение роботов по оптимальному пути к конечным точкам, не столкнувшись с препятствиями.

1. Поле разделено на $p \times q$ квадратных ячеек.
2. Препятствия подразделяются на статические и динамические.
3. Динамические препятствия – другие роботы.
4. Линейная скорость робота выражена константой.
5. Цель алгоритма – обеспечить следование эталонному пути, исключить столкновения с препятствиями.

Реализация алгоритма
дописана

Выводы

Поставленная цель и задачи были выполнены. Результатом моей работы стал приятный, минималистичный язык для изучения алгоритмов с инструментами для сборки, форматирования и расширения кодовой базы.

Литература

1. Ахо А. В., Лам М. С., Сети Р., Ульман Дж. Д. Компиляторы: принципы, технологии и инструменты. — 2-е изд. — М. : Вильямс, 2008.
2. Нистром Р. Создание интерпретаторов / пер. с англ. — М. : ДМК Пресс, 2022.
3. Книт Д. Э. Искусство программирования. Т. 1: Основные алгоритмы. — 3-е изд. — М. : Вильямс, 2006.
4. Уэлч Т. А. Метод сжатия данных на основе словаря // ТИИЭР. — 1984. — Т. 72, № 6.

Литература

5. Пратт Т. У., Зелковиц М. В. Языки программирования: разработка и реализация. — 4-е изд. — СПб. : Питер, 2002.
6. Себеста Р. У. Концепции языков программирования. — 10-е изд. — М. : Вильямс, 2015.
7. Грune D., Jacobs C. J. H. Parsing Techniques: A Practical Guide. — 2nd ed. — New York : Springer, 2008.
8. Scott M. L. Programming Language Pragmatics. — 4th ed. — Amsterdam : Morgan Kaufmann, 2015.

Разработка языка программирования Weasel



Студент группы ПС-31
Глухарев Степан

Научный руководитель
Нехорошкова Людмила Георгиевна